

Schedule-Free On-Demand Water Transport in Bergen

September 9, 2026

Abstract

This report documents the design of the real-time dispatch logic for a small-boat, on-demand water transport service in Bergen, Norway. The service does not exist yet: rather than optimizing an existing system, the project builds a model of how such a service would operate, replacing fixed timetables with a dispatch policy that recomputes itself every few minutes. Six pieces are covered: a navigable routing module that computes real, coastline-respecting travel times between the four candidate stops; a synthetic demand model anchored in official Norwegian population and employment data; a simulation environment, implemented to the Gymnasium interface standard, that hosts the dispatch policy; a fixed, rule-based reference policy, including the fleet-coordination logic needed to run it with more than one boat, verified against that environment at three increasing scales; a reinforcement-learning agent trained with proximal policy optimization to learn a dispatch policy directly, including the sequence of reward-shaping experiments that determined how the agent is trained; and a final comparison of the trained agent against the reference policy, aggregated over held-out evaluation seeds and examined in detail on one of them.

Contents

1	Introduction and Motivation	3
2	Related Work	3
3	Navigable Routing	3
4	Demand Model	5
4.1	Catchment areas and masses	5
4.2	From population and employment to origin-destination intensity	6
4.3	From intensity to Poisson arrival rates	7
4.4	Arrivals and group sizes	8
4.5	Persons vs. groups	9
5	Simulation Environment	9
5.1	State	9
5.2	Actions	10
5.3	Transition	10
5.4	Reward	10
6	Reference Dispatch Policy	11
6.1	Decision rule	11
6.2	Coordinating several free boats at once	11
6.3	Verification at three scales	12
6.4	Known limitations	15

7	Reinforcement Learning Agent	15
7.1	Method	16
7.2	Implementation, observation, and action	16
7.3	Reward function	17
7.4	Reward-shaping experiments	18
7.5	Fresh demand on every reset	19
7.6	Training run	19
8	Results: Agent vs. Reference Policy	21
8.1	Aggregate comparison, five seeds	21
8.2	Detailed comparison on a single seed	22
9	Assumptions and Limitations	23
10	Conclusion	24

1 Introduction and Motivation

Bergen is a coastal city whose public transport network is structured as a hub-and-spoke system centered on the city center, a layout imposed by the surrounding fjords and mountains. As the city grows and tourism increases, this network is under increasing pressure, and land-based expansion is constrained by the same geography that makes it congested in the first place. The fjords that divide the city are, at the same time, an underused transport corridor.

This project explores a small-boat, on-demand water transport service as a complementary mode. Two principles guide its design. First, the service is not modeled as a profit-making venture (public transport of this kind is rarely self-financing, and its justification is environmental and related to congestion relief, not revenue); the objective is to deliver a good level of service efficiently, not to maximize income. Second, the flexible layer of the service has no fixed timetable: a passenger requests a trip and a boat is dispatched to serve it, subject to a waiting-time guarantee, and the dispatch policy is recomputed at short, regular intervals rather than following a pre-published schedule. Section 2 situates this "schedule-free" operating philosophy, and the static water-taxi model it builds on, within the closely related literature.

Scope of this report. It covers the routing and demand modeling that feed the project (Sections 3 and 4), the simulation environment that hosts the dispatch problem (Section 5), a fixed reference dispatch policy verified against that environment (Section 6), a reinforcement-learning agent trained to learn a dispatch policy directly (Section 7), and a final comparison between the two (Section 8). The environment is intentionally simple (four stops, direct point-to-point trips, a small set of rules), chosen so that its mechanics could be verified before adding complexity to the model of how boats and passengers behave. Section 9 states this and the project's other simplifying choices explicitly, together, rather than leaving them implicit.

2 Related Work

Three papers anchor the modeling choices described in this report.

Gu and Wallace [2021] formulate a static location, fleet-sizing, and routing model for a water-taxi system connecting the same four stops used in this project (Kleppestø, Laksevåg, Bryggen, and Sandviken). It is the closest precedent in terms of the physical instance, and it is used in Section 4 as one reference point for the scale of daily demand.

Braathen et al. [2024] study autonomous ferry scheduling under labor regulations, comparing autonomous and manually operated fleets. The demand-generation approach in Section 4 is adapted from their multiplicative, gravity-style formulation, and the group-size statistics they report are used to calibrate the group-size distribution used here.

Sánchez-Martínez et al. [2016] develop a schedule-free operating paradigm for high-frequency transit control. Their central point, that abandoning fixed schedules in favor of real-time, resource-aware control can improve service quality provided the underlying control problem can be solved effectively, is the conceptual starting point for treating dispatch in this project as a control problem rather than a scheduling problem.

3 Navigable Routing

Travel times between the four stops are computed from a navigable-routing module rather than from straight-line (Haversine) distances. An earlier, straight-line version of the travel-time matrix was found, on visual inspection, to draw lines that cross land near Bryggen, which sits inside the Vågen bay next to the Nordnes peninsula (straight-line distances are therefore not usable directly and are corrected here).

Water/land classification. A basemap tile mosaic covering the four stops is downloaded at zoom level 14, which corresponds to a fixed Web Mercator resolution of $156543.03392/2^{14} \approx 9.55$ m/pixel. Because Web Mercator distorts distances by a factor of $1/\cos(\text{latitude})$, and Bergen sits at approximately 60.4°N , the real ground resolution is $9.55 \times \cos(60.4^\circ) \approx 4.7$ m/pixel. Each pixel is classified as water or land by its Euclidean distance, in RGB space, to a reference water color sampled from the basemap style (threshold of 15 color units); the resulting mask is pixel-for-pixel aligned with the source map, since both are produced by the same Web Mercator projection formula and the same Earth radius.

Graph and shortest paths. Each water pixel becomes a node in an 8-connected graph (4 orthogonal and 4 diagonal neighbors), with edge weights equal to the real distance between pixel centers; land pixels are simply absent from the graph, so no edge can cross land. The four stops are snapped to the nearest water pixel, restricted to the graph’s main connected component, to avoid anchoring a stop to an isolated puddle unconnected to open water. A multi-source Dijkstra search (`scipy.sparse.csgraph.dijkstra`) then computes the shortest navigable path between every pair of stops in a single call, exact because all edge weights are non-negative real distances.

Design speed and resulting matrix. Rather than calibrating vessel speed against an existing ferry line, a fixed design speed of 30 km/h (≈ 16.2 knots) is used for the whole fleet, treated as a design decision for the small on-demand vessels rather than an empirical calibration. Table 1 reports the resulting travel-time matrix, and Table 2 compares it against the straight-line matrix it replaces: routing around land adds between 6% and 24% to travel distance depending on the pair, with Laksevåg (whose corrected location sits further inside the Puddefjorden) showing the largest correction because the detour around the Nordnes peninsula is large relative to its own distance to the other stops.

Table 1: Navigable travel-time matrix (minutes), 30 km/h design speed.

	Kleppestø	Laksevåg	Bryggen	Sandviken
Kleppestø	-	5.3	11.5	9.6
Laksevåg	5.3	-	8.6	8.6
Bryggen	11.5	8.6	-	6.8
Sandviken	9.6	8.6	6.8	-

Table 2: Straight-line vs. navigable travel times, all six stop pairs.

Pair	Straight-line (min)	Navigable (min)	Difference
Laksevåg ↔ Bryggen	7.0	8.6	+23.8%
Kleppestø ↔ Sandviken	8.7	9.6	+10.5%
Bryggen ↔ Sandviken	6.3	6.8	+8.9%
Kleppestø ↔ Laksevåg	4.9	5.3	+7.8%
Kleppestø ↔ Bryggen	10.7	11.5	+7.3%
Laksevåg ↔ Sandviken	8.1	8.6	+6.4%

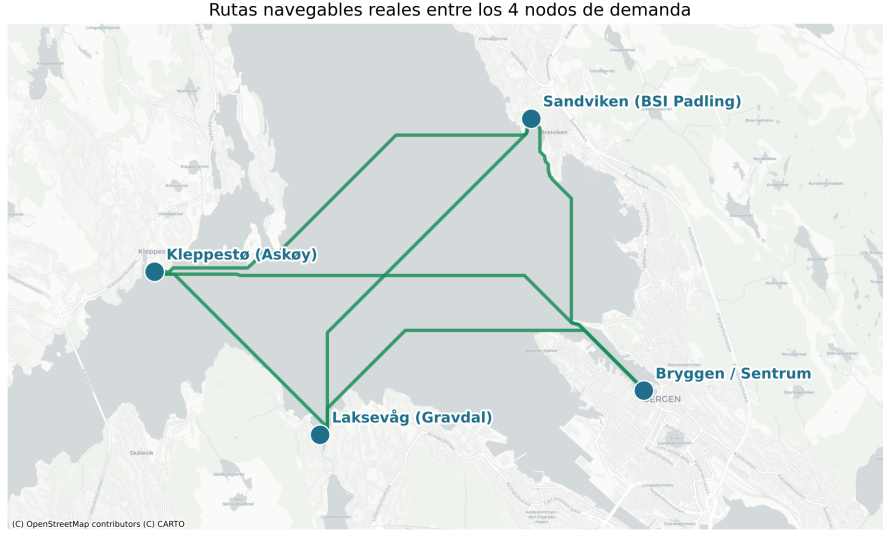


Figure 1: The six navigable routes between the four demand stops, computed by Dijkstra search over the water-classified pixel graph.

The known limitation of this method is that an 8-connected grid only offers eight travel directions per step, so a shortest path can zig-zag slightly instead of following a perfect diagonal when the true direction falls between two of the eight allowed ones; the resulting overcost is a few percent in the worst case and does not affect the central conclusion that the method reliably avoids land.

4 Demand Model

No real demand data exists for this service, since it does not operate yet. Demand is therefore synthesized, but anchored in real open data wherever possible.

4.1 Catchment areas and masses

Population and employment are read from official Statistics Norway (SSB) 250-metre grids. For each of the four stops, a catchment area was hand-drawn by the project team in QGIS; every *grunnkrets* (Norway’s smallest statistical reporting unit) that the drawn area touches, even partially, is included in full for that stop, and duplicate assignments at shared boundaries are resolved by keeping each grunnkrets with the stop it overlaps the most. Population and employment are then summed over the SSB grid cells inside the resulting area. Table 3 reports the resulting masses, and Figure 2 shows which grunnkretser were assigned to each stop.

Table 3: Catchment area, population, and employment by node (SSB grids, 2026).

Node	Grunnkretser	Area (km ²)	Population	Employment
Kleppestø (Askøy)	28	80.6	25,153	7,550
Laksevåg (Gravdal)	18	14.9	15,986	7,760
Bryggen / Sentrum	67	20.2	27,695	52,213
Sandviken (BSI Padling)	21	14.6	11,109	4,106

Las 4 zonas de captación (grunnkretser elegidas a mano)

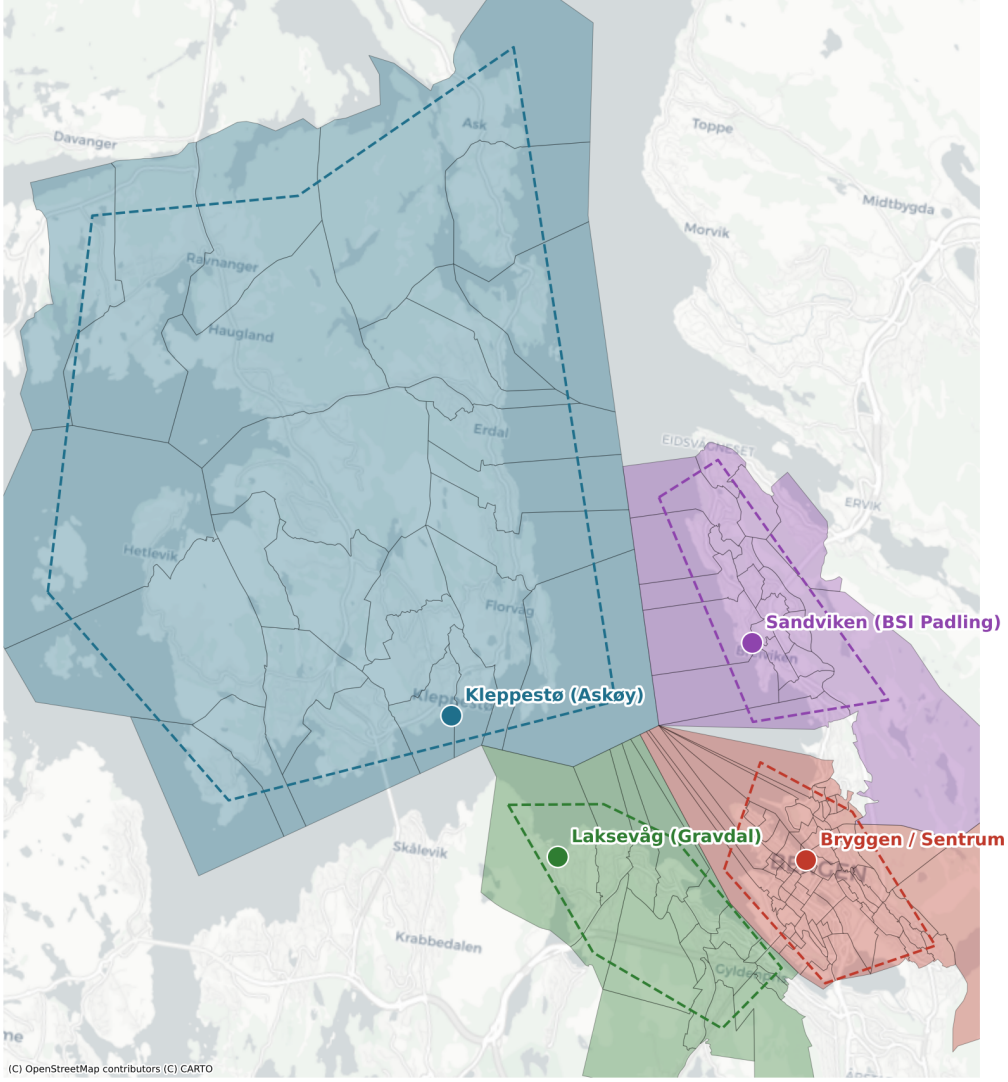


Figure 2: The four hand-drawn catchment areas (dashed outline) and the grunnkretser assigned to each node (filled).

4.2 From population and employment to origin-destination intensity

Braathen et al. [2024] generate synthetic demand using a multiplicative popularity score that combines different factors associated with the attractiveness of stations, travel times, weekdays, and hours. This project adopts the general logic of using a score to represent the relative attractiveness or intensity of demand, but replaces the judgment-based popularity factors with observable population, employment, and university-related measures for the Bergen study area. The score used in this project is defined as follows.

Let $\text{pop}(n)$ and $\text{emp}(n)$ be the population and employment of node n from Table 3, and let $u(n) \in [0, 1]$ be the university weighting factor described below. Masses are first normalized against the largest node, so that both scales end up comparable before being multiplied:

$$\text{pop}_{\text{norm}}(n) = \frac{\text{pop}(n)}{\max_m \text{pop}(m)}, \quad \text{emp}_{\text{norm}}(n) = \frac{\text{emp}(n)}{\max_m \text{emp}(m)} \cdot u(n). \quad (1)$$

$u(n)$ compensates for the fact that registered employment does not count students, and Bergen is a university city; no open source of fine-grained enrollment data was found, so $u(n)$ is fixed

by hand for now (1.0 at Bryggen and Sandviken, reduced at Kleppestø and Laksevåg), declared here as a design choice rather than measured data. A third, "mixed" mass averages the two, used at times of day with no clear commute direction:

$$\text{mix}(n) = \frac{1}{2}(\text{pop}_{\text{norm}}(n) + \text{emp}_{\text{norm}}(n)). \quad (2)$$

Each of the four time bands in Table 4 assigns a role (population, employment, or mixed) to the origin side and to the destination side of every trip; write $w(n, \text{role})$ for whichever of the three masses above that role selects. The origin-destination intensity for band b and the ordered pair (o, d) , $o \neq d$, is then

$$I_b(o, d) = w(o, \text{role}_{\text{origin}}(b)) \cdot w(d, \text{role}_{\text{destination}}(b)) \cdot f_{\text{dist}}(o, d) \cdot v(b), \quad (3)$$

where $v(b)$ is the band's relative volume factor (Table 4) and f_{dist} is a distance-decay term not present in Braathen et al. [2024]'s formulation, added in this project:

$$f_{\text{dist}}(o, d) = \exp\left(-\frac{t(o, d)}{\tau}\right), \quad \tau = 15 \text{ min}, \quad (4)$$

with $t(o, d)$ the navigable travel time from Table 1. $I_b(o, d)$ has no unit of its own; it is a relative weight, only meaningful compared to the other 11 pairs of the same band, and is turned into an actual passenger count in the next step.

Table 4: Time bands, relative demand volume, and origin/destination roles.

Band b	Hours	$v(b)$	Role _{origin} / Role _{destination}
Morning	06-09	1.00	population / employment
Midday	09-15	0.40	mixed / mixed
Afternoon	15-18	1.00	employment / population
Evening	18-24	0.15	mixed / mixed

4.3 From intensity to Poisson arrival rates

Every hour of the day inherits the intensity of the band it belongs to (a step function across bands, not a smoothed hourly curve): $I_h(o, d) := I_{b(h)}(o, d)$ for hour h in band $b(h)$. This is computed for all 24 operating hours and all 12 ordered pairs, covering all four bands, not a subset.

The day's total passenger budget is set as a percentage of the population of the four catchment zones of Table 3 (not the total population of Bergen and Askøy), scaled down on weekends:

$$S = p \cdot P_{\text{zonas}} \cdot f_{\text{semana}}, \quad P_{\text{zonas}} = \sum_n \text{pop}(n), \quad (5)$$

where p is `demanda.porcentaje_poblacion_dia` in the demand model's configuration file (currently 10%, an explicit, changeable team parameter, not a value taken from either reference paper), $P_{\text{zonas}} = 79,943$ is the sum of the population column of Table 3, and f_{semana} is 1.0 on weekdays and 0.4 on weekends. S is a number of *passengers*; dividing by the mean group size $\bar{g} = 8.6$ (Section 4.4) gives the day's total number of *groups*, $G = S/\bar{g}$, which is then distributed across all hour-pair cells in proportion to their share of the day's total intensity:

$$\lambda(o, d, h) = \frac{I_h(o, d)}{\sum_{h', o', d'} I_{h'}(o', d')} \cdot G. \quad (6)$$

Within each hour, the number of arriving groups for pair (o, d) is drawn $N \sim \text{Poisson}(\lambda(o, d, h))$, and each group's arrival minute is drawn uniformly within that hour.

As a reference point for the order of magnitude of S : Gu and Wallace [2021], who study the same four stops, assume 300 travel requests per day in their case study. That number is for a water-taxi service, a different kind of offering (booked, point-to-point, presumably priced and scheduled differently from a public-transport-style service), so it is not directly comparable to S here; it is mentioned only because it is a verifiable, same-instance data point, not as a validation of the 10% figure.

4.4 Arrivals and group sizes

Passenger arrivals within each hour follow the Poisson process described above. Braathen et al. [2024] report a mean group size of 8.6 passengers with standard deviation 2.6, but do not specify a sampling distribution for it (their own group size is a deterministic function of the popularity score pop_{pg} , not a random draw). Since group size is a count of people, a discrete, non-negative distribution is used instead of a continuous one: a Binomial($n = 40, p = 0.215$), calibrated by matching moments to Braathen et al.’s reported mean and standard deviation, which reproduces the target mean exactly (8.60) and the target standard deviation closely (2.598 against 2.6).

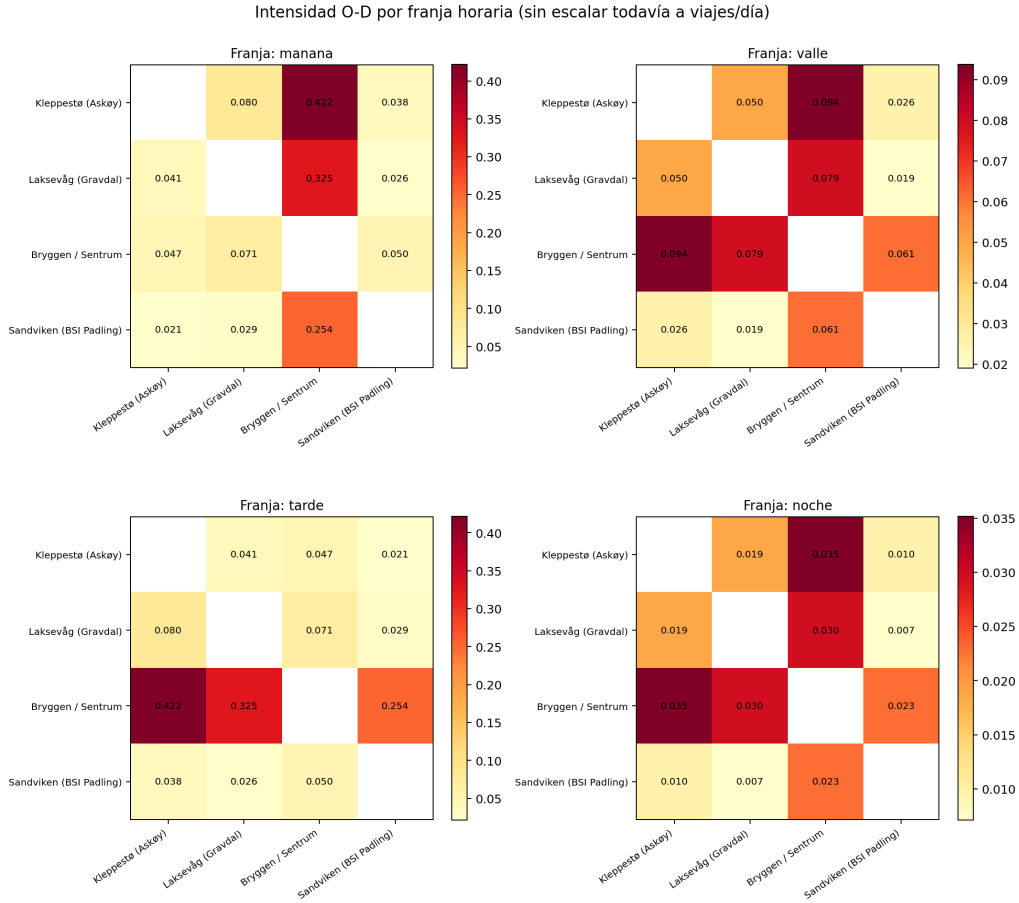


Figure 3: Origin-destination intensity $I_b(o, d)$ by time band. Morning traffic concentrates toward Bryggen; the afternoon band reverses the pattern.

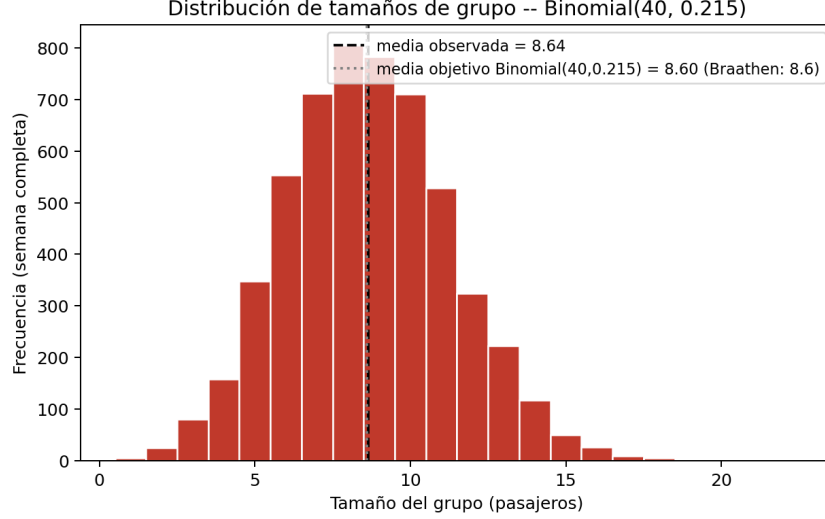


Figure 4: Observed group-size distribution over a simulated week, against the target Binomial(40, 0.215).

4.5 Persons vs. groups

A group, as generated above, is the atomic unit of the demand model. For the dispatch simulation environment (Section 5), each group can optionally be exploded into individual persons, each inheriting the same origin, destination, and arrival time as the group they came from; this changes the atomic unit seen by the simulator’s capacity and boarding logic without altering when or where demand appears.

5 Simulation Environment

This is a first, simple iteration. The environment described in this section establishes a working structure for the dispatch problem, not the final model the project targets. It is deliberately minimal: four stops, direct point-to-point boat trips, and a small set of rules, chosen so that the environment itself (its mechanics, its bookkeeping, its interface) could be verified before adding any complexity to the model of how boats and passengers behave. Extensions are expected as the project continues.

The dispatch problem is modeled as a Markov decision process and implemented as a discrete-time simulation environment following the Gymnasium interface standard (`reset()` / `step()`), which is not a library of pre-built simulators but an interface contract: the environment implements the entire boat-and-passenger logic itself, and only its shape is standardized, so that a reinforcement-learning library can later drive it without modification. The environment advances in fixed 2-minute steps, rather than jumping between discrete events, for simplicity and because it maps directly onto the `step()` call.

5.1 State

The state has three parts. For each boat: origin node, destination node (equal to the origin if idle), minutes remaining until arrival, current occupancy, and whether it is free to receive a new order. For demand: one queue per origin-destination pair (twelve pairs for four nodes, not one queue per node), holding the number of people waiting and the waiting time of the oldest person in that specific pair. For time: minute of day and day of week. This rich, dictionary-shaped state is used for logging and inspection; a separate fixed-size numeric vector is derived from it for the learning agent (Section 7.2 gives its exact composition and size), built from one-hot node

encodings for each boat’s origin and destination, a normalized time-to-arrival and occupancy per boat, normalized counts and waiting times per pair queue, and two plain normalized scalars for time (minute of day divided by the length of the day, day of week divided by six) rather than a cyclical encoding: every simulated run is one self-contained episode that starts at a fixed hour and ends within the same day, so the wraparound a cyclical encoding exists to smooth out (23:59 rolling into 00:00, Sunday into Monday) never actually occurs inside a single episode, and a plain linear value carries the same information with two entries instead of four. Two fields present in the dictionary-shaped state are deliberately left out of this numeric vector: whether a boat is free is redundant once its origin and destination one-hot encodings and its time-to-arrival are already present (free is exactly the case origin equals destination and time-to-arrival is zero), and including it as a separate entry would only repeat information already available to the network in a different form.

5.2 Actions

Every 2 minutes, each free boat receives one of five orders: travel to one of the four nodes, or wait. A boat already en route ignores any new order and continues to its destination; it is not redirected mid-trip. Boarding follows a fixed rule, not chosen by the policy: when a free boat at node A is ordered to travel to node B , it boards strictly from the queue for the pair (A, B) , in arrival order, up to capacity, and travels directly to B , where everyone aboard disembarks. Trips are therefore always direct, point-to-point journeys between exactly two stops; a boat never carries passengers with mixed destinations, since it can only ever board from the one queue that matches the trip it was just ordered to make. Ordering a free boat to travel to its own current node is treated identically to the explicit wait order: both leave the boat’s destination equal to its origin, so neither boards anyone, advances the boat, nor incurs the per-step movement penalty of Section 7.3; the event log used for the metrics reported throughout this document records both forms of staying put under the same label, so that waiting time is never undercounted simply because a policy expressed it as ”go to where I already am” rather than the dedicated wait action.

5.3 Transition

Each step executes, in order: (1) apply the chosen action to every free boat, boarding from the matching pair queue as described above; (2) advance all boats in transit by the fixed time step, disembarking everyone aboard any boat that reaches its destination; (3) add to the queues any new arrivals whose arrival time falls within the interval just elapsed (these do not board the boat that just departed in step 1, since they had not yet arrived when boarding happened); (4) compute the step’s reward and assemble the new state.

5.4 Reward

The reward returned at every 2-minute step is a single negative scalar that penalizes letting people wait or travel beyond a tolerance and, more lightly, sending a boat under way. This scalar plays no role in the reference dispatch policy of Section 6, which follows a fixed rule and never consults it; it exists purely to give the environment a well-defined objective for a policy that has to learn from experience. Because the reward is meaningful chiefly in that training context, and because its exact shape (in particular, whether the discomfort term is capped, and how strongly movement is penalized) is itself a design choice explored deliberately rather than fixed in advance, its full definition and the reasoning behind each term are given together in Section 7.3, next to the experiments it is meant to explain.

6 Reference Dispatch Policy

Before any policy is asked to learn, it is useful to have a simple, fixed rule that can be verified by hand and used as a point of comparison. This section describes that rule, a coordination problem that surfaced only once several boats used it together, and the results of running it on three test instances of increasing size.

6.1 Decision rule

The reference policy, referred to here as nearest-available, decides one free boat at a time and follows a single principle: serve the demand waiting at the boat’s own current stop before ever considering a trip to reposition toward demand elsewhere. Concretely, when boat b is free at node A , the policy first looks at the twelve pair queues (Section 5.1) restricted to those leaving A ; if any of them holds someone waiting, the boat is sent to the destination whose queue has been waiting longest, without regard to how many people are in it. Only when no one at all is waiting to leave A does the policy consider repositioning: it looks across every other pair queue in the system and sends the boat, empty, toward the origin node of whichever queue has waited longest, so that the boat can pick that demand up once it arrives.

This rule is deliberately myopic. It never looks two moves ahead: a boat ordered to reposition simply travels to the node with the oldest waiting demand and re-evaluates from scratch once it gets there, rather than planning the pickup and the eventual delivery as a single two-leg trip. It is also strictly local-first, which is a real design choice with a real cost. Consider a boat that becomes free at Bryggen just as five people who arrived a minute earlier ask to go to Laksevåg, while a single person has already been waiting twenty minutes at Kleppestø for a boat to Sandviken. Nearest-available sends the boat to the five people at Bryggen, not to the person who has waited far longer at Kleppestø, because it never compares urgency across nodes while there is still anyone to serve locally. This keeps the rule simple to state and to verify, and it avoids abandoning demand the boat could serve immediately in favor of chasing something that still requires travel time; the cost is that a lightly connected node can, in principle, be left waiting for a long time if the busier nodes never run dry. Because the environment no longer removes anyone from a queue for having waited too long (Section 5.3), this is not a theoretical risk that resolves itself: nothing in the environment stops that wait from growing arbitrarily long if the policy never gets around to it. Whether this actually happens in practice is exactly what the verification below is designed to check.

6.2 Coordinating several free boats at once

A second issue appears only when more than one boat is free at the same node in the same step. Applying the decision rule above independently to each boat, using the same snapshot of the world for both, can make two boats choose the same destination: each one sees, say, eleven people waiting at a shared destination and decides that is the most urgent trip to make, without knowing the other boat just made the identical decision. The first boat to be processed picks up all eleven; the second arrives at the same destination with nobody left to serve, having paid the cost of the trip for nothing. This was not a hypothetical concern. It was found by inspecting the state of a verification run at a single simulated minute and confirmed in the accompanying event log, and it visibly hurt the fraction of demand served in that run.

The fix, applied whenever more than one boat is free in the same step, is to assign boats to actions one at a time rather than all at once from the same snapshot. Boats are processed in a fixed order; after each one is assigned a destination, the pair queue it would draw from is reduced locally, in a scratch copy of the state, by exactly what that boat would pick up, following the same first-in-first-out and capacity rule the environment itself uses when boarding. The next boat in line then applies the decision rule to this already-discounted picture of demand, so

a second boat heading to the same busy destination sees only what would genuinely still be waiting for it, not what the first boat already claimed. This coordination step does not change the underlying decision rule; it only prevents two boats from acting on the same stale piece of information. All results reported below use this coordinated version, referred to as *fleet assignment*, never the single-boat rule applied independently to a group. The combination of strict local-first prioritization (Section 6) with this one-at-a-time fleet coordination is what makes nearest-available a competent reference policy rather than a trivial one: it is simple to state, but reasoning correctly about several simultaneously free boats sharing the same demand picture is not automatic, and Section 8 shows it setting a bar that is not trivial for a learned policy to clear.

6.3 Verification at three scales

The coordinated policy was run on three test instances built from the demand model of Section 4, each scaled down from the model’s own 10% daily participation rate so that a run stays easy to follow while still exercising the full mechanics of the environment. Table 5 summarizes the three.

Table 5: The three verification instances: scale, fleet, and purpose.

	Step 1	Step 2	Step 3
Time window	Morning, 06:00 to 09:00	Full day, 06:00 to 24:00	Full week (7 days)
Boats	2	3	3
Groups / people	23 / 202	133 / 1106	620 / 5429
2-minute steps	90	540	7 × up to 540
Purpose	Step-by-step visual check	Aggregate metrics, one day	Weekday/weekend pattern

Step 3 is run as seven independent one-day episodes, one per weekday, rather than as a single continuous week-long clock; the demand generator already applies its own weekday/weekend intensity factor (Table 4 does not show this factor, which multiplies the whole day’s volume by 0.4 on Saturdays and Sundays), so each day already carries the right pattern, and running seven separate days avoids having to decide what an idle boat should do overnight, when no demand exists in the model between hours 0 and 6.

Conservation check. Every person the demand generator creates must be accounted for at the end of a run, either as served, as still waiting in a queue, or as still on board a boat that had not yet reached its destination when the run ended. This identity, generated equals served plus still waiting plus still aboard, was checked automatically on all three runs and held exactly in every case, which is a basic but necessary sanity check given that the environment tracks people individually rather than as aggregate counts.

Table 6: Aggregate results of the reference policy at the three scales.

Metric	Step 1	Step 2	Step 3
Percent served	100.0%	100.0%	99.6%
Mean waiting time (min)	17.4	10.9	10.0
Mean / maximum time in system (min)	27.2 / 54.0	20.5 / 53.6	19.8 / 57.1
Waiting time, 50th / 90th / 95th percentile (min)	15.6 / 33.4 / 33.5	9.4 / 21.6 / 26.3	9.1 / 21.4 / 25.1

Table 6 reports the headline numbers. All three runs serve essentially everyone; the small shortfall in Step 3 (23 people out of 5429, none of them actually lost, just still on board a boat when one of the seven daily windows closed) reflects the mechanical fact that seven independent one-day cutoffs give seven separate chances for a boat to be mid-trip exactly when the clock

runs out, not a failure of the policy. Because nothing in the environment discards a slow case anymore (Section 5.3), the percentiles reported here are the real, uncensored distribution of how long a served person actually waited, including its tail, rather than a distribution that quietly excludes whoever would have taken longest.

Step 2, despite carrying 5.5 times the demand of Step 1 with only one additional boat, reports a shorter mean wait and shorter tail percentiles. This is not a contradiction: Step 1 concentrates its entire demand into three peak hours with no slack period for a backlog to clear, while Step 2 spans eighteen hours with alternating busy and quiet bands, giving the fleet repeated opportunities to catch up between peaks.

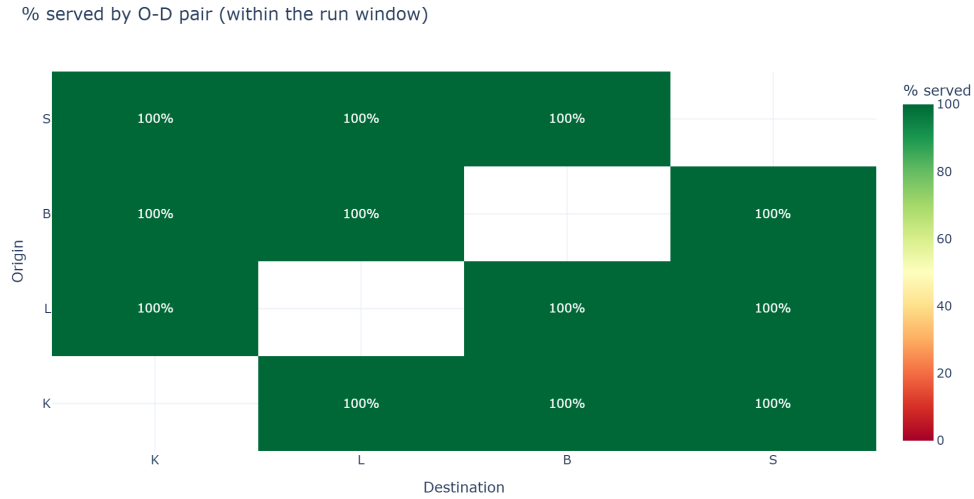


Figure 5: Percent of demand served per origin-destination pair, Step 2 (full day). K, L, B, and S denote Kleppestø, Laksevåg, Bryggen, and Sandviken.

Fleet occupancy over time

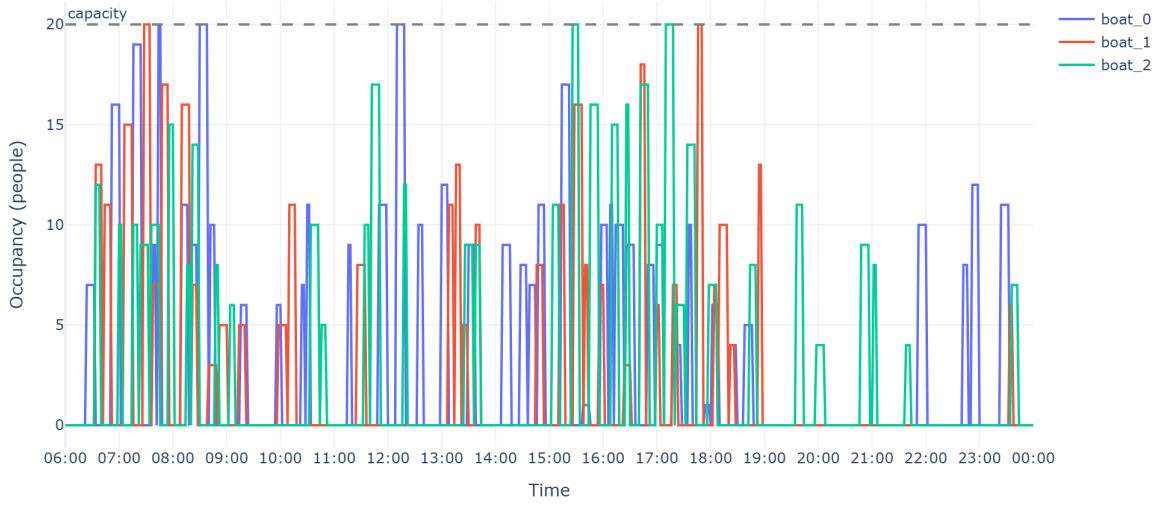


Figure 6: Occupancy of each boat over the full day, Step 2. Both boats spend a large share of the day near full capacity during the morning and afternoon commute bands.

People waiting over time

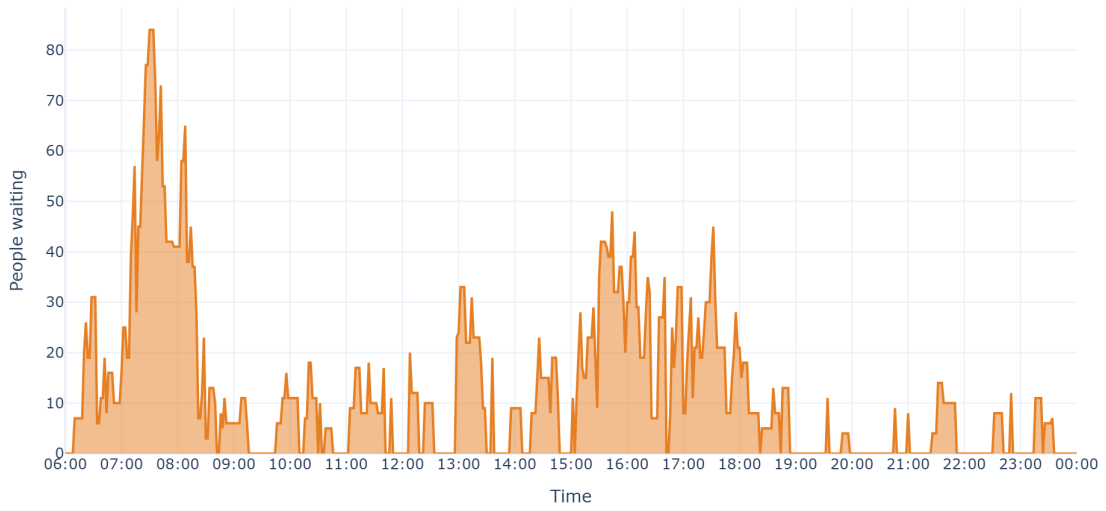


Figure 7: Number of people waiting, summed over all twelve pair queues, over the course of Step 2. The two peaks correspond to the morning and afternoon commute bands of Table 4.



Figure 8: Six snapshots of the fleet over the course of Step 2, taken at even intervals from 06:00 to the following midnight. Orange labels show how many people are waiting at a stop at that instant; a full step-by-step, minute-by-minute view is available as an interactive, browser-based visualization produced by the same code, not reproducible as a static figure here.

Figure 5 breaks the aggregate percent-served figure down by origin-destination pair and confirms that every one of the twelve pairs reaches 100% in Step 2, not just the total. Figure 6 and Figure 7 show, respectively, how full each boat runs and how many people are waiting system-wide as the day unfolds; both track the two commute peaks of the demand model closely, which is the expected behavior of a policy that reacts to demand rather than anticipating it. Figure 8 gives a static sense of the fleet in motion; the underlying tool can also produce an animated, minute-by-minute version and a version that can be stepped through one decision at a time, both used during development to visually confirm the fleet-assignment fix of Section 6.2, but neither reproduces meaningfully on a printed page.

6.4 Known limitations

Three limitations of this policy are worth stating explicitly rather than leaving implicit. First, the strict local-before-remote priority described above means a lightly connected pair can in principle wait indefinitely while a busier pair keeps the fleet occupied; this did not occur in the three runs reported here, but nothing in the rule itself prevents it, and a busier or more imbalanced instance could expose it. Second, fleet assignment (Section 6.2) prevents two boats from claiming the same destination in the same step, but it still assigns boats one at a time, in a fixed order, rather than searching over every possible joint assignment for the one that would serve the most people; it removes the worst failure mode without claiming to be optimal. Third, repositioning is myopic by construction, evaluating only the immediate move rather than the pickup and delivery as a planned pair, so a boat can occasionally reposition toward demand that a different, better-timed choice would have served faster.

7 Reinforcement Learning Agent

The reference policy of Section 6 is a fixed rule with no parameters to tune; it cannot improve with experience, and it does not attempt to plan more than one move ahead. This section describes replacing it with a policy that learns: the learning method and why it fits this problem (Section 7.2), how the environment is observed and acted on (Section 7.2), the reward the policy is trained to optimize and the deliberate sequence of experiments that shaped its exact form

(Sections 7.3-7.4), and the training run itself (Section 7.6), with results given together with the reference policy’s in Section 8.

7.1 Method

The learning method used is proximal policy optimization, or PPO [Schulman et al., 2017], an on-policy, actor-critic reinforcement-learning algorithm. In broad terms, PPO alternates between two phases: it runs the current policy in the environment to collect a batch of experience (states visited, actions taken, rewards received), and it then updates the policy’s parameters to make the actions that led to above-average outcomes more likely and the actions that led to below-average outcomes less likely, using that same batch. What distinguishes PPO from earlier policy-gradient methods is a clipped objective function that keeps each update close to the policy that generated the data, which in practice makes training considerably more stable: an update that would change the policy too abruptly in a single step is capped rather than applied in full. This matters here because the environment is stochastic (a fresh demand realization every episode, Section 7.5) and the reward is comparatively sparse and delayed relative to the two-minute decision interval, both conditions under which more aggressive update rules tend to destabilize training.

PPO was chosen over value-based alternatives such as deep Q-networks specifically because of the action space described in Section 5.2: one of five discrete choices per boat, which for a fleet of n_b boats is a `MultiDiscrete` space of size 5^{n_b} , not a single flat discrete choice. The standard deep Q-network implementation in the software library used for training (Section 7.2) does not support `MultiDiscrete` action spaces natively, while PPO’s policy network naturally outputs one independent categorical distribution per boat and requires no adaptation to handle several simultaneous discrete decisions.

7.2 Implementation, observation, and action

The agent is implemented with Stable-Baselines3 [Raffin et al., 2021], a maintained, widely used reinforcement-learning library built on PyTorch, driving the same Gymnasium-standard environment [Brockman et al., 2016] described in Section 5; no separate implementation of the environment exists for training, since the whole point of standardizing the environment’s interface in the first place was to let a learning library drive it without modification. The policy is Stable-Baselines3’s default `MlpPolicy`, a small feedforward network with two hidden layers of 64 units each and hyperbolic-tangent activations, shared as a common trunk between the policy head (which outputs the five-way action distribution for each boat independently) and the value head (which estimates expected future reward from the current state, used internally by PPO to judge whether a batch of experience was better or worse than expected).

The network’s input is exactly the fixed-size numeric vector described in Section 5.1: for each boat, a one-hot encoding of its origin node, a one-hot encoding of its destination node, a normalized time-to-arrival, and a normalized occupancy (ten entries per boat, with four nodes); for each of the twelve ordered pair queues, a normalized count of people waiting and a normalized oldest-waiting-time (twenty-four entries); and two plain normalized scalars for time, minute of day and day of week (two entries). For the two-boat, four-node instance used for training (Step 1 of Section 6.3), this totals 46 entries. The network never sees the rich, dictionary-shaped state used for logging and inspection, and it never sees anything beyond this vector: no privileged information about, for instance, which specific person is waiting where, only the aggregated counts and times that the vector encodes.

7.3 Reward function

PPO trains a policy to maximize expected cumulative reward. The reward defined below is built as a cost: it is zero or negative at every step, so maximizing it is the same thing as minimizing that cost. We state it as a maximization only because that is the convention PPO and the rest of the RL literature use, not because the underlying goal is to maximize anything. What we actually want is to keep people from spending a long time in the system and to avoid moving boats without reason. That trade-off is written as one number because PPO, like essentially every policy-gradient method, needs a single scalar to optimize; it cannot be handed two separate objectives. Turning several objectives into one weighted sum is a standard step in this kind of optimization problem, usually called scalarization, and the two paragraphs after the formula explain how the weight was chosen here, not just declare it.

All the results in this report use `unidad_demanda` = "personas" (Section 4), so every unit the simulator tracks is one individual person; the formula below is written directly in terms of persons rather than the more general "person or group" wording used elsewhere in the code, since group size never actually appears in anything trained or evaluated here.

Let t index the 2-minute simulation steps, $\tau_{\text{tol}} = 12$ minutes the comfort tolerance (the longest direct navigable trip, Table 1), and $\mathcal{A}_t$ the set of people active at step t , meaning waiting in a queue or already aboard a boat. For each person $i \in \mathcal{A}_t$, let $s_{i,t}$ be the time they have spent in the system as of step t , and define their surplus as the part of that time past the tolerance:

$$\text{surplus}_{i,t} = \max(0, s_{i,t} - \tau_{\text{tol}}). \quad (7)$$

Their discomfort penalty is a quadratic function of that surplus, normalized by a fixed width $\sigma = 18$ minutes so that the result is a plain number, not a quantity in minutes:

$$\text{penalty}_{i,t} = \left(\frac{\text{surplus}_{i,t}}{\sigma} \right)^2. \quad (8)$$

Dividing by σ before squaring is what makes $\text{penalty}_{i,t}$ dimensionless: minutes over minutes cancel, leaving a ratio (how many "widths" of surplus this person has accumulated), and squaring a dimensionless ratio is still dimensionless. The square itself is deliberate: someone a minute past the tolerance barely registers, someone thirty minutes past it registers a lot more than thirty times as much. The penalty is meant to grow faster than the wait itself, so the optimizer feels increasing pressure to deal with the worst cases, not just the average one. Equation 8 has no cap: this is the version actually used to train and evaluate the agent reported in the rest of this document. A capped variant, $\min(C, \cdot)$, was tested and rejected; Section 7.4 shows why and reports that experiment.

Let m_t be the number of boats under way at step t . The full reward is

$$r_t = - \left[\sum_{i \in \mathcal{A}_t} \text{penalty}_{i,t} + w_{\text{move}} \cdot m_t \right], \quad (9)$$

with w_{move} read from configuration, not hard-coded. m_t , like the penalty, is a plain count with no unit; w_{move} is what gives it a cost in the same currency as discomfort. Table 7 shows how fast the discomfort term grows.

Table 7: Discomfort penalty by surplus, one person, uncapped (Equation 8).

Situation	Penalty
Waiting 12 minutes or less (within tolerance)	0
18 minutes total (6 minutes of surplus)	$(6/18)^2 \approx 0.11$
24 minutes total (12 minutes of surplus)	$(12/18)^2 \approx 0.44$
30 minutes total (18 minutes of surplus)	$(18/18)^2 = 1.00$
60 minutes total (48 minutes of surplus)	$(48/18)^2 \approx 7.11$

Picking w_{move} : matching scales, not guessing. Table 7 gives the scale to match: a person who has just gone one minute past the tolerance already costs about 0.003 per step. If w_{move} were, say, 0.1 (the value used in production, where this reward is never actually optimized: Section 6.3 runs the reference policy on a different, simpler formula that has no bearing on training), a single boat trip would cost more than thirty people who are each barely over tolerance, and the agent would have every incentive to leave people waiting rather than move a boat. So the search for w_{move} started from that ≈ 0.003 figure and tried values at and below it, which is what Section 7.4 reports.

This is also why the reward used for training and evaluation is a different formula from the one used to report the reference-policy runs in Section 6.3: this reward exists only to give PPO something to optimize, and its shape only matters in that context. Applying it to the reference policy would not change how that policy behaves (it is a fixed rule that never looks at the reward), and applying the production formula to the agent would defeat the point of the experiments below.

7.4 Reward-shaping experiments

Two things about the reward were undecided before spending the long training run of Section 7.6: whether capping the discomfort penalty helps or hurts a learning agent, and how large w_{move} can be before the agent starts avoiding boat trips. Both were tested directly rather than assumed. Each configuration below trained for 30,000 timesteps (a few minutes on Step 1) and was then evaluated against the reference policy on the same five held-out seeds used in Section 8.

Capped vs. uncapped discomfort (A vs. B). Configuration A uses the capped penalty,

$$\text{penalty}_{i,t} = \min\left(C, \left(\frac{\text{surplus}_{i,t}}{\sigma}\right)^2\right), \quad C = 1.0, \quad (10)$$

with $w_{\text{move}} = 0$. Configuration B removes the cap ($C \rightarrow \infty$, i.e. Equation 8 with no modification) and is otherwise identical. The reasoning behind testing this: with the cap active, someone who has waited 35 minutes and someone who has waited 50 minutes are both charged the same maximal penalty, so the reward stops distinguishing between them past that point: an agent chasing reward has no more incentive to help the person at 50 minutes than the one at 35. Removing the cap keeps that distinction alive for as long as someone is unserved, so the hypothesis was that B should give the agent a standing incentive to go after whoever has been waiting longest, not treat everyone past the cap as equally urgent.

Adding a small movement penalty (C1 vs. C2). Starting from the better of A and B, this pair adds w_{move} back in, at the two candidate values discussed above: C1 sets $w_{\text{move}} = 0.003$, C2 sets $w_{\text{move}} = 0.001$, to see whether a smaller weight still shapes movement without costing service.

Table 8: Reward-shaping experiments: configuration and outcome after 30,000 training timesteps, evaluated on five held-out seeds.

Configuration	Cap C	w_{move}	Percent served	Mean wait (min)	Movements
A	1.0	0	47.8%	26.2	135
B	∞	0	51.9%	21.9	125
C1	∞	0.003	52.0%	23.8	127
C2	∞	0.001	44.7%	22.5	95

Removing the cap (B) beats the capped configuration (A) on every column, including movements, so B is the base carried into C1/C2. Between those two, C1 tracks B closely on percent served while moving about as much; C2, with a smaller weight, both serves fewer people and moves less, and the fewer movements it does make are not used any better than C1’s. *Percent served is read here only as a rough dial for comparing configurations against each other within this short-training sequence, not as the metric that matters: Section 8 explains why and reports mean and maximum time in system instead once the real training budget is spent.* Neither run here gets close to the reference policy, but 30,000 timesteps is a fifth of what Section 7.6 uses for the agent actually evaluated later, so that is expected: the point of this sequence is choosing a reward shape cheaply, not producing a competitive agent yet. The combination carried forward is the uncapped penalty with $w_{\text{move}} = 0.003$ (C1).

7.5 Fresh demand on every reset

Every run reported in Section 6.3 uses one fixed table of demand, generated once and reused for the whole run, which is the right choice for verifying that the environment and the reference policy behave correctly: the policy has no parameters to overfit, so there is nothing to gain from varying the demand across runs. Training a learning agent on a single fixed table of demand is a different matter and a real risk: with enough experience against exactly the same arrivals, at exactly the same simulated minutes, a large enough network can begin to memorize that particular realization rather than learn a policy that generalizes to the underlying arrival pattern it is drawn from. To avoid this, the training environment regenerates its demand table on every reset, drawing a new realization of the same gravity-and-Poisson process of Section 4 each time, so that every training episode is an independent Monte Carlo sample of the same demand pattern rather than a repeat of the previous one. A reset called with an explicit seed still reproduces that exact realization, which is what makes the fixed evaluation seeds of Section 8 comparable across runs.

7.6 Training run

The reward-shaping choice of Section 7.4 (uncapped discomfort, $w_{\text{move}} = 0.003$) is carried into a single, longer training run of 150,000 timesteps on the Step 1 instance, roughly five times the budget used for any individual configuration in that exploratory sequence and the only long training run performed in this project; every other configuration is deliberately short, so that this one run is the point at which training budget itself, rather than reward shape, is the variable being spent.

A handful of further training settings are fixed identically across every run reported in this section and the exploratory sequence before it: the network observes and is rewarded on normalized quantities (via running mean and variance statistics computed online, frozen at their trained values for evaluation, Section 8) rather than on raw values, an entropy coefficient of 0.01 (above Stable-Baselines3’s default of zero) to keep exploration from collapsing too early, and a learning rate of 3×10^{-4} , the standard PPO default. The rollout length, meaning how many environment steps PPO collects before each policy update, is 512, smaller than Stable-Baselines3’s own default of 2048 so that updates happen roughly every five to six 90-step episodes rather than every twenty-plus, but 512 itself does not come from a formula tied to this problem, only from an earlier round of tuning. The more direct number to defend is the episode length itself, 90 steps, so that every update sees a whole number of complete episodes rather than cutting one in half; both are compared below.

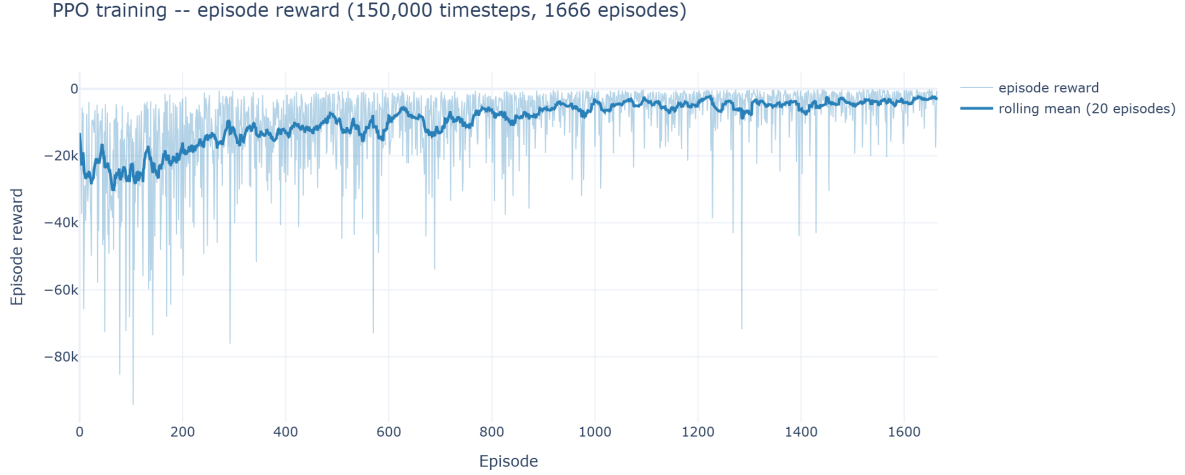


Figure 9: Episode reward over the course of the 150,000-timestep training run, $n_{\text{steps}} = 512$ (raw reward per episode and a 20-episode rolling mean).

Mean episode reward improved from $-24,871.4$ over the first 20 episodes of training to $-2,832.8$ over the last 20, almost an order of magnitude, with the final individual episodes ($-17,540.6$, $-1,905.5$, -651.3 , $-1,007.3$, -958.0) mostly settled into a low range punctuated by occasional worse episodes, consistent with the fresh, unseen demand realization drawn on every reset (Section 7.5) rather than with unstable training.

Checking the rollout length: $n_{\text{steps}} = 90$. The same 150,000-timestep run was repeated with every setting identical except n_{steps} , set to 90 instead of 512, to see whether aligning updates to episode boundaries changes the result. On the training data the two look similar: mean reward over the last 20 episodes is $-2,832.8$ for $n_{\text{steps}} = 512$ against $-2,956.2$ for $n_{\text{steps}} = 90$, close enough that the training curve alone would not obviously favor either. Evaluated on the same five held-out seeds used in Section 8, the two diverge sharply:

Table 9: $n_{\text{steps}} = 512$ vs. $n_{\text{steps}} = 90$, same reward and same training budget, evaluated on the five held-out seeds.

Metric	$n_{\text{steps}} = 512$	$n_{\text{steps}} = 90$	Reference policy
Mean time in system (min)	29.0	30.2	26.1
Maximum time in system (min)	60.8	73.7	69.7
Mean waiting time (min)	17.1	19.9	16.4
Movements (5 episodes)	191	179	155
Reward (5 episodes)	-3772.0	$-28,082.5$	-5259.0
Percent served (context only)	88.0%	77.5%	88.0%

$n_{\text{steps}} = 90$ loses on every column, including the two that matter most: its worst case (73.7 minutes) is worse than the reference policy’s, not just worse than $n_{\text{steps}} = 512$ ’s, and its evaluation reward ($-28,082.5$) is close to where $n_{\text{steps}} = 512$ started training, not where it ended. Training performance and evaluation performance pulled apart for this configuration even though both runs looked similar while training. The most likely reason: with $n_{\text{steps}} = 90$ the rollout buffer holds exactly one episode, so PPO updates about 1,666 times over the run (once per episode) instead of about 293 times, each update built from an order of magnitude less experience. More frequent updates on smaller batches make each gradient step noisier and more

tied to that one episode’s particular random demand, which can look like steady improvement during training while leaving the policy less robust to demand it has not seen. $n_{\text{steps}} = 512$, despite not being chosen for a principled reason either (Section 7.6), ends up giving each update a wider, less noisy sample to learn from, and that turns out to matter more than lining updates up with episode boundaries. The agent used throughout Section 8 is the $n_{\text{steps}} = 512$ run; $n_{\text{steps}} = 90$ is reported here as a result that did not hold up under evaluation, not adopted.

8 Results: Agent vs. Reference Policy

The trained agent of Section 7.6 and the reference policy of Section 6 are run on the same five evaluation seeds, 1001 through 1005, chosen outside the range of seeds used to generate training episodes so that neither policy is evaluated on demand it could have seen before. Because both policies act on the same `EntornoDemandaAleatoria.reset(seed=...)` call (Section 7.5), a given seed produces the identical demand table for both, so any difference in outcome reflects the policy, not the demand.

Why percent served is not the headline metric. Percent served is reported below for context, but it is not what either policy is actually being judged on, and it should not be read as the main result. The environment does not lose anyone (Section 5): a person who has not boarded a boat by the time a run ends is not gone, they are simply still in a queue or still aboard when the clock stops, and would be served if the run kept going a bit longer. So “percent served” mixes two different things: how well the policy dispatches boats, and how much of the operating window happened to be left when the last few people arrived, which is arbitrary and has nothing to do with policy quality. The two things the reward (Section 7.3) actually penalizes, and the two things worth comparing here, are how long people spend in the system, mean and especially worst case, and how many times the fleet moved to achieve that.

8.1 Aggregate comparison, five seeds

Table 10: Agent vs. reference policy, aggregated over the five evaluation seeds. Reward is computed with the training reward (Section 7.4) for both policies, so that the total is comparable between them.

Metric	PPO agent	Reference policy	Difference
Mean time in system (min)	29.0	26.1	+3.0
Maximum time in system (min)	60.8	69.7	−9.0
Mean waiting time (min)	17.1	16.4	+0.6
Total movements (5 episodes)	191	155	+36
Total reward (5 episodes)	−3772.0	−5259.0	+1487.0
Percent served (context only, see above)	88.0%	88.0%	0.0 pp

The agent beats the reference policy on the worst case (60.8 vs. 69.7 minutes, the number the uncapped reward pushes on directly) and on the reward it was trained against, at the cost of a higher mean time in system and 36 more boat movements over the five episodes. That trade-off is the direct, expected consequence of removing the discomfort cap (Section 7.4): with no ceiling on any one person’s penalty, the agent keeps a standing incentive to stop any single case from growing into a very long wait, even if that means accepting a slightly worse average and moving boats more to chase down the worst cases. Percent served happens to come out identical for both, 748 of 850 people over the five seeds. That is interesting mainly because it means the two are not being compared on wildly different operating points, not, per the paragraph above, evidence of anything about service quality on its own.

The result is not uniform across seeds. The agent’s reward is worse than the reference policy’s on three of the five (1001, 1003, 1005), and the aggregate is carried by seed 1004, where the reference policy has its single worst run of the ten (-4091.1) while the agent handles the same demand comfortably (-1525.6). That is consistent with an agent that is more consistent in its worst case rather than uniformly better episode by episode. Cross-checking the decision log directly: the agent chooses to wait in only 5.4% of its decisions (11 of 202), against 47.5% for the reference policy (140 of 295). The extra movements above are not idle waste: the agent is genuinely dispatching more actively, not just spending the movement penalty on nothing.

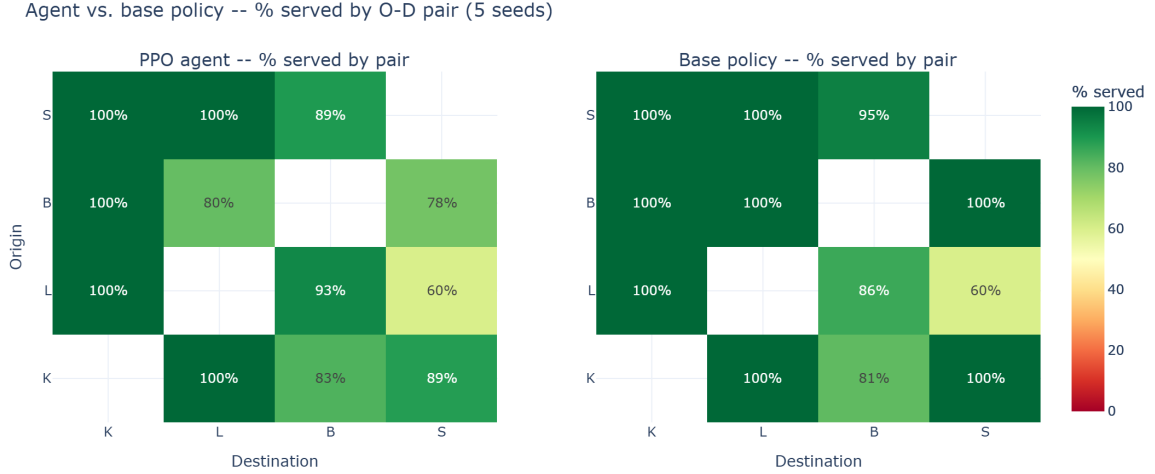


Figure 10: Percent served by origin-destination pair, agent vs. reference policy, aggregated over the five evaluation seeds.

8.2 Detailed comparison on a single seed

To see behavior, not just the aggregate, seed 1001 is examined with the same package of metrics and figures produced for the reference-policy verification of Section 6.3 (waiting profile, fleet occupancy over time, compliance heatmap by pair, reward breakdown, and backlog by pair), computed separately for the agent and for the reference policy on that one seed, so that the concrete pattern of a single run is visible side by side for both.

On this specific seed the reference policy has a clearly shorter mean wait than the agent (9.4 vs. 17.4 minutes), which is the opposite of what the aggregate reward comparison above might suggest. This is why the detailed view matters: seed 1001 is one of the three individual seeds where the reference policy’s reward is better than the agent’s, and the aggregate advantage comes from the reward being sensitive to the worst case across all five seeds, not from the agent doing better seed by seed. Both readings are correct at the same time: the agent is the stronger choice by the worst-case-aware objective it was trained on, and the reference policy can still be the better choice on a representative single run like this one. (Percent served happens to move the same direction here too, 124 of 135 for the reference policy against 110 of 135 for the agent, but per the caveat above that is not the reason to prefer either policy on this seed; the wait-time gap is.)

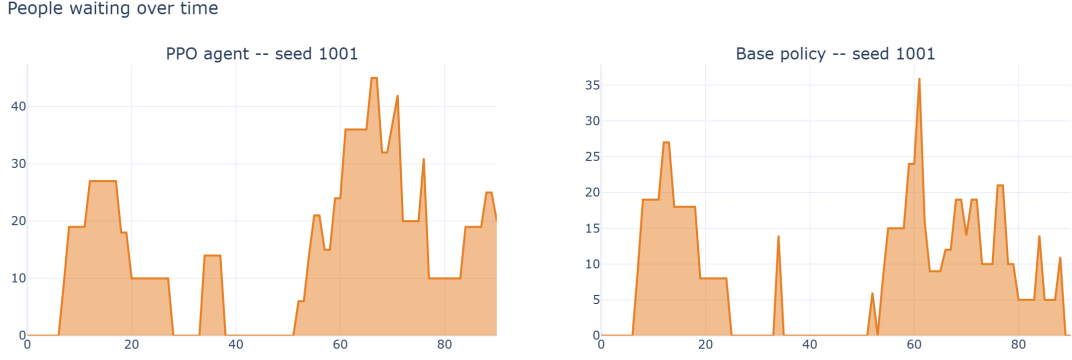


Figure 11: People waiting over time, seed 1001: agent (left) vs. reference policy (right).

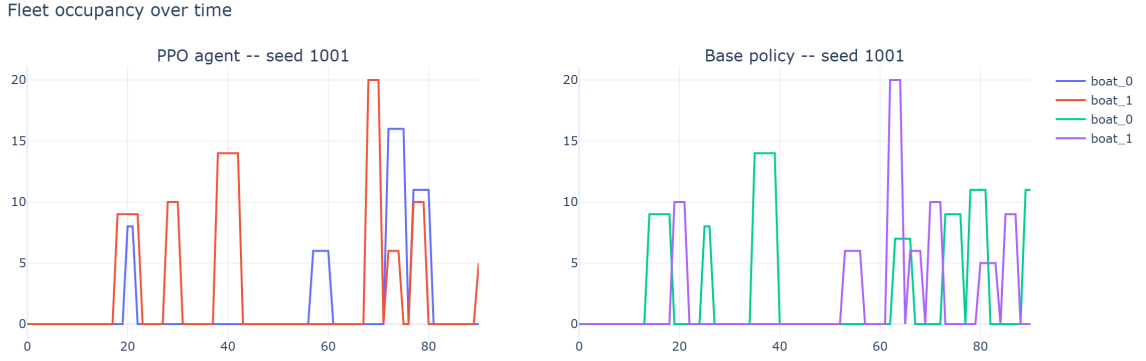


Figure 12: Fleet occupancy over time, seed 1001: agent (left) vs. reference policy (right).

9 Assumptions and Limitations

This project makes a number of explicit simplifying choices, stated here together rather than left scattered or implicit.

No real demand data. The service does not exist, so no observed origin-destination counts are available anywhere in this report; Section 4’s gravity-style intensity score, its distance-decay term, its role assignment by time band, and its daily participation rate are all team decisions made in the absence of that data, anchored in real population and employment figures but not validated against ridership.

No passenger patience, by design. The environment does not remove anyone from a queue for having waited too long (Section 5); every person generated is either served or still in the system when a run ends. This was a deliberate choice to obtain uncensored waiting-time statistics (Section 6.3), but it also means neither policy has any pressure resembling a hard service-level deadline: the reference policy never consults the reward at all, and the agent

is trained on the uncapped discomfort penalty (Section 7.3), which grows without bound the longer someone waits rather than plateauing.

The reference policy is local-first and myopic (Section 6.4). It is a considered heuristic, not an optimizer, and the comparison in Section 8 should be read as agent-vs-heuristic, not agent-vs-optimal.

The agent is trained and evaluated on a single, small instance. Both the reward-shaping experiments of Section 7.4 and the long training run of Section 7.6 use Step 1 (two boats, the morning window) exclusively; nothing in this report establishes how the trained agent would behave on the larger Step 2 or Step 3 instances, or on a fleet of a different size.

Reward shape was explored, not exhaustively searched. Section 7.4 tests four configurations along two design axes (discomfort cap, movement weight) at one training budget each; other combinations, and other training budgets for the same combinations, were not tried, so the chosen configuration is a reasonable, evidence-based choice rather than a demonstrably optimal one.

10 Conclusion

This project set out to build the simulation and dispatch logic for a schedule-free, on-demand water transport service in Bergen, and to compare a fixed reference policy against a learned one on that simulation. The fixed policy, nearest-available with fleet coordination (Section 6), serves as more than a placeholder: verified at three increasing scales, it serves essentially all demand (99.6-100%) and its coordination step (Section 6.2) is what makes it work at all with more than one boat sharing the same demand picture. The reinforcement-learning agent, trained with PPO after a deliberate, evidence-based sequence of reward-shaping experiments (Section 7.4) rather than by assumption, beats the reference policy on the metric its reward is built around, the worst case of time in system (60.8 vs. 69.7 minutes), at the cost of a slightly longer mean wait and a more active fleet (Section 8). The seed-level detail in the same section is a deliberate corrective to reading the aggregate table alone: on a single representative run, the reference policy can wait noticeably less than the agent, so which policy is preferable depends on whether the objective is a shorter typical wait or protection against the worst individual case. This is a question this project surfaces with concrete evidence rather than resolves.

The most direct next steps follow from the limitations of Section 9: training and evaluating the agent on the larger Step 2 and Step 3 instances, to see whether its behavior generalizes beyond the two-boat morning instance used here; extending the reward-shaping search beyond the four configurations tested; and, longer term, using the uncensored waiting-time percentiles this environment already produces (Section 6.3) to propose a concrete service-time guarantee, the kind of commitment a real schedule-free service would need to state and hold itself to.

References

- Caroline Braathen, Julio C. Goez, and Mario Guajardo. Autonomous ferries in light of labor regulations – a passenger perspective. *Maritime Transport Research*, 7:100115, 2024. doi: 10.1016/j.martra.2024.100115.
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. Openai gym. *arXiv preprint arXiv:1606.01720*, 2016.

- Yong Gu and Stein W. Wallace. Operational benefits of autonomous vessels in logistics – a case of autonomous water-taxis in Bergen. *Transportation Research Part E: Logistics and Transportation Review*, 154:102456, 2021. doi: 10.1016/j.tre.2021.102456.
- Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.
- Gabriel E. Sánchez-Martínez, Nigel H. M. Wilson, and Haris N. Koutsopoulos. Schedule-free high-frequency transit operations. *Public Transport*, 9:285–305, 2016. doi: 10.1007/s12469-016-0129-8.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.